

Evaluation Report

Farmer RAG Agricultural Advisory Agent

Date: February 19, 2026

A. Architecture Evolution

HW2 Baseline → HW3 Evolution

The HW2 system used a linear pipeline with a single-agent flow where the LLM processed queries, optionally called tools (weather, market, knowledge base), and responded sequentially: Query → Retrieve → Generate → Respond. State was minimal (current query + tool outputs), with no structural separation between planning, execution, and verification.

HW3 refactored this into a role-based multi-node architecture using LangGraph with three purpose-built nodes:

- Planning Node (reasoning_node): Applies ReAct-style reasoning with structured JSON output to decide the next action or final response.
- Execution Node (tool_executor_node): Invokes tools and records results cleanly into shared agent state.
- Critique Node (verification_node): Checks response groundedness by extracting claims and cross-referencing retrieved sources.

Graph structure: START → reasoning → [execute_tool | verify] → [reasoning | END]. We changed because this gives us modularity (each node evolves independently), adaptive control (conditional routing without touching core logic), full observability (failures attributable to a specific node), and easy extensibility for new tools or verification heuristics.

B. Memory Design Rationale

Two-Tier Memory Model

The system uses two tiers: short-term AgentState holds in-flight data for the current turn (tool results, retrieved sources, verification outputs) and is discarded after response generation. Persistent storage in PostgreSQL holds everything of lasting value: full conversation and message history, user profiles (farms, crops, location), tool call logs, and advisories with groundedness scores and source citations.

This design enables cross-session continuity (farmers return days later and pick up where they left off), personalization (stored profiles eliminate re-asking for background), auditability (tool calls and advisories can be reviewed post-hoc), and cost control (only final outcomes are persisted, not every intermediate reasoning step).

Write Policy: Messages are saved after every exchange; tool calls and advisories are committed after each agent run. On load, full history is fetched to restore conversational context.

Pruning (docs/memory_strategy.md): Conversations are archived after 90 days; threads exceeding 50 messages are summarized; older tool call records are aggregated with recent ones retained for analysis.

C. Metrics & Results

Evaluation Metrics

- Groundedness Score: Fraction of claims supported by retrieved sources (0–1).
- Tool Selection Accuracy (F1): Expected vs. actual tool invocations.
- Task Completion Rate: Binary ,querying: did the agent produce a substantive, on-topic response? (0 or 1)
- Iterations Before Convergence: Reasoning steps taken before final response.
- Hallucination Frequency: Ratio of unsupported claims to total claims.

Results — February 19, 2026

ID	Query	Category	Ground.	Tool Acc.	Comple.	Iter.	Pass
kb_001	Recommended spacing for maize?	knowledge	1.00	1.00	1.00	2	✓
kb_002	Recommended spacing for beans?	knowledge	0.50	1.00	1.00	2	✗
kb_003	Recommended spacing for tomatoes?	knowledge	0.50	1.00	1.00	2	✓
kb_004	What is the weather in Kigali today?	weather	1.00	1.00	1.00	2	✓
kb_005	When to harvest maize and how to dry it?	knowledge	1.00	1.00	1.00	3	✓
complex_001	Tell me about maize planting area requirements.	complex	1.00	1.00	1.00	2	✓

Summary: 5 passed, 1 failed | Avg Groundedness: 0.83 | Avg Tool Accuracy: 1.00 | Avg Completion: 1.00 | Avg Hallucination Freq: 0.17

Test cases were drawn from sample_ingest.txt and TAS 4402-2010. The single failure (kb_002) scored 0.50 groundedness due to insufficient source coverage for bean spacing. Tool accuracy and task completion were perfect across all six cases.

D. Failure Case Deep Dive

Failure: 'object of type NoneType has no len()'

What Happened: When a user sent a follow-up question ("How can I do weed control?") after a weather query, the agent returned an error instead of advice, fully breaking conversational continuity.

Why It Happened: Three code paths unsafely called `len()` on values that could be `None`:

(1) `state.get('retrieved_sources', [])` returns `None` when the key exists with a `None` value and its not the default `[]`, causing a crash in the verifier's groundedness footer. (2) The verification LLM sometimes returned a JSON wrapper object instead of a bare list; iterating worked until `len(verified_claims)` hit `None`. (3) The tool executor's `state.get` pattern caused `None + sources` errors when `retrieved_sources` was explicitly set to `None`.

How We Fixed It: Replaced all `state.get('key', [])` with `state.get('key') or []` to normalize `None`. Added `or []` guards after `_extract_claims` and `_verify_claims`. Added `if not isinstance(verifications, list): verifications = []` before iteration. Guarded `.extend()` calls on messages, tool_calls, and retrieved_sources against `None` in state merge logic.

What Improved: Follow-up questions no longer crash the agent. Conversational continuity across topic shifts works reliably. Defensive patterns are applied consistently, and the verifier handles any malformed LLM output gracefully.

E. Scalability Reflection

1,000 Concurrent Users

The database connection pool and synchronous tool execution would fail first. Each request holds an open DB connection while blocking on LLM inference, retrieval, and verification which are three latency-heavy operations. At 1,000 concurrent users, connection exhaustion stalls new requests, and the vector search layer becomes a secondary bottleneck. Mitigations: Enforcing PgBouncer connection pooling, async I/O refactor, read replicas, caching for frequent queries, and rate limiting on the LLM and retrieval tiers.

High-Stakes Domain such as Pesticide Dosages, Crop Failure

Groundedness guarantees break down first. The verification pipeline catches many unsupported claims but cannot guarantee correctness since a hallucinated dosage could cause direct, material harm. Mitigations: applying stricter groundedness thresholds with automatic escalation, mandatory citation requirements, domain-specific validation for example cross-checking dosage values against known safe ranges, and human-in-the-loop review for high-risk recommendations.

Regulated Industry (e.g., FDA, Agricultural Bodies)

Auditability and reproducibility fail first. Regulators require a tamper-evident record of which model version produced an output, which documents informed it, and the full reasoning trail. Current logs help but don't constitute a full audit trail — prompts and retrieval logic are not formally versioned per release. Mitigations: immutable append-only audit logs, versioned prompts and model identifiers, full input-output-source capture per advisory, and compliance-grade data retention with access controls.

4. Contribution Statement

Team Member	Technical Contributions	Evaluation / Debugging Contributions
Sibomana Glorry	Implemented role-based graph architecture	Designed test cases
Sibomana Glorry	Implemented the reasoning node, tool executor	Ran evaluation suite, analyzed failure modes
Kwagalakwe Ethel	Built evaluation metrics, verification node, test suite	Debugged NoneType failure, added defensive checks, improved prompts
Kwagalakwe Ethel	Database schema, memory persistence, chat API integration	Verified cross-session memory, validated DB outputs and advisory logs